

Урок 16. Корутины. Отмена - это ошибка.

19 июня 2020

В этом уроке обсуждаем как работает отмена корутин под капотом

Отмена

Урок про отмену корутин уже был. Там мы в основном рассмотрели, как правильно реагировать на отмену. А сейчас уже знаний достаточно, чтобы понять, как отмена работает под капотом, и что будет происходить с родительской и дочерними корутинами в этом случае. Этот урок можно считать небольшим дополнением.

Когда мы вызываем метод `cancel`, корутина переходит в состояние `cancelling` (из-за этого `isActive = false`). О своей отмене она уведомляет родителя и дочерние корутин. При этом, под капотом используются те же механизмы уведомления, что и при возникновении ошибок. Но вместо реальной ошибки, корутина при отмене создает специальную фейковую - `CancellationException`. Эту ошибку корутина шлет **родительской** и **дочерним** корутинам в уведомлениях о своей отмене. Рассмотрим их реакцию на это уведомление.

Родительская корутина проверяет, что за ошибку ей прислали. Если бы там пришла реальная ошибка, то корутина отправила бы ее дальше - своему родителю и отменила бы саму себя и свои дочерние корутин. Но в случае, когда приходит `CancellationException`, родительская корутина никак не реагирует.

Дочерняя корутина ничего не проверяет. Если родитель отменился, то и ей тоже надо отменяться, независимо от типа ошибки. Это принцип `structured concurrency`. Поэтому она отменяет себя и шлет уведомления об отмене дальше вниз, своим дочерним корутинам, если они есть.

Т.е. при отмене корутина отменяет все свои дочерние и далее вниз. А вверх отмена не идет.

suspend функции

Отдельно имеет смысл рассказать про поведение `suspend` функции при отмене корутины.

`suspend` функции бывают **обычные** и **отменяемые** (`cancellable`). Обычные мы создаем оператором `suspendCoroutine`, а отменяемые - `suspendCancellableCoroutine`. Если вы используете стороннюю `suspend` функцию, то по идее у нее в документации должно быть указано, отменяемая она или нет.

Обычные `suspend` функции не обращают внимания на то, что корутина отменилась. Они работают в обычном режиме и возвращают результат.

А **отменяемые** `suspend` функции при старте подписываются на отмену корутины. Когда происходит отмена корутины, `suspend` функция получает об этом уведомление с `CancellationException` и сразу шлет этот `Exception` в `Continuation`, как результат своей работы.

В итоге вызов `suspend` функции в корутине выбрасывает `CancellationException` и работа корутины на этом останавливается (если `suspend` функция не была обернута в `try-catch`).

Сама `suspend` функция при этом продолжает работать и даже может потом отправить результат в `continuation.resume`. Но этот результат уже будет никому не интересен.

Чтобы прервать работу `suspend` функции в случае отмены вызвавшей ее корутины, нам доступен специальный колбэк `invokeOnCancellation`:

```
1 suspend fun myCancellableSuspendFunction(): String {
2     return suspendCancellableCoroutine {continuation ->
3
4         continuation.invokeOnCancellation {
5             // ...
6         }
7
8         // ...
9     }
10 }
11 }
```

Он будет вызван сразу, как только корутина будет отменена. И в нем мы сможем попытаться отменить уже запущенную асинхронную работу.